FIT1056 – Introduction to software engineering
PST5

The Project Journey: An Overview - Problem-solving tasks 5

We will use the case study of a Music School Management System (MSMS) to introduce the fundamental software engineering (SE) concepts from the perspective of a developer. Before you begin, review the **Applied#1** slides to make sure you understand how to work with the Git repository.

Music School Management System (MSMS) (Individual Task)

Objective: You will build a Music School Management System (MSMS) over five stages. Each stage is a direct upgrade of the previous one, taking you from a simple script to a robust, professional application.

Important Notes:

- This is an **individual task**.
- You are **not** required to create a formal Software Requirements Specification (SRS) document.
- You **must** use Git to track your progress. You will make a new "commit" after completing each part.

- **PST1: The Foundation.** Build a simple, in-memory prototype.
- **PST2: The Upgrade.** Add file storage, data validation, and better organization.
- **PST3: The Architecture.** Rebuild with a professional Object-Oriented (OOP) design.
- **PST4: The User Interface.** Replace the text console with a modern Graphical User Interface (GUI).
- **PST5: The Quality Assurance.** Make the application "crash-proof" and prove it works with automated tests.

Part 0: Project Setup and Git Initialization

Same Git steps of PST1 is mentioned here. You may make the changes accordingly as you want.

Goal: Create a project directory and initialize it as a Git repository. This is the foundation for your entire project.

Make sure you are on the correct branch (individual). If you have any un-committed changes, commit them now and push. Feel free to push after each individual commit, there is no problem with doing so.

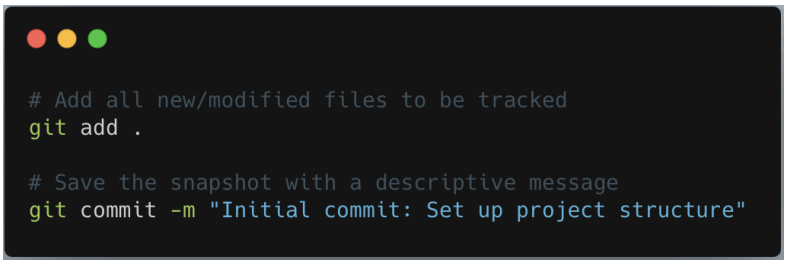
Your Task:

1. Open your terminal or command prompt (Follow the Applied-Week01 Slides).
2. Create a new folder for your project (e.g., msms-project) and navigate into it.
3. Initialize a new Git repository in this folder. This command creates a hidden .git subdirectory that will track all your changes.





4. Create your main source code file. (e.g., main.py).
5. Add this new file to the Git staging area and make your first commit. A commit is a snapshot of your code at a specific point in time.

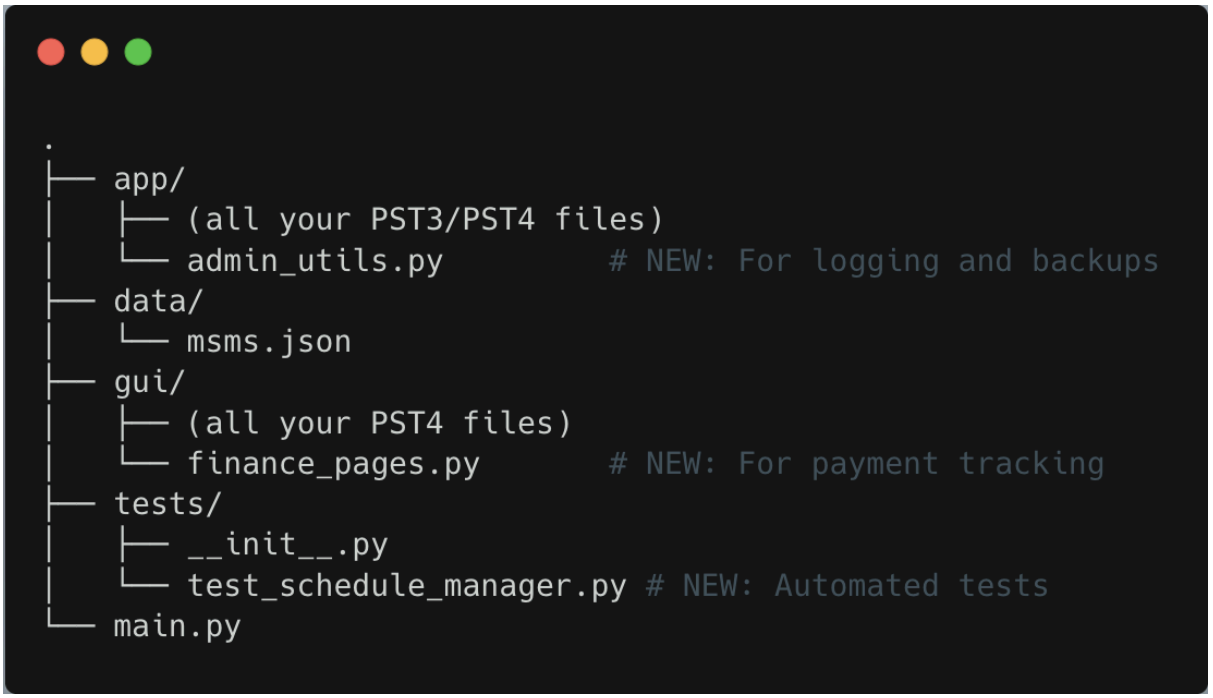


Part 5 (PST5): The Quality & Finance Gate

Your Goal: To solve the "Fragile & Unverified" problem. This final stage transforms your application into a professional-grade product. You will add

critical business features like financial tracking, robust administrative tools like logging and backups, and a suite of automated tests to ensure the application is reliable and correct.

The New Directory Structure: We add a directory for tests and a file for new admin utilities.



Fragment 5.1: The Finance & Reporting Engine

- **Mission:** To implement the core business logic for financial transactions and exporting data reports directly within your central controller.
- **Your Task:**
    1. Open your ScheduleManager class in app/schedule.py.
    2. In the __init__ and _load_data methods, add support for a new self.finance_log list.
    3. Create the record_payment(student_id, amount, method) method, which will add a new payment dictionary to the finance log.
    4. Create the get_payment_history(student_id) method to retrieve all transactions for a single student.
    5. Create the powerful export_report(kind, out_path) method to write financial or attendance data to a CSV file.
- **Check the Template in (Fragment5_1.py and add to app/schedule.py)**

Checkpoint: Commit Your Progress
Save your file and commit the change.

```
git add app/schedule.py
git commit -m "feat(finance): Implement core finance tracking and CSV reporting logic"
```

(Note: "Feat" is a common convention for a commit that introduces a new feature.)

Fragment 5.2: The GUI for Finance

- **Mission:** To expose the new financial features through the user interface.
- **Your Task:**
  - Create the file gui/finance_pages.py.
  - Implement the show_finance_page(manager) function.
  - This function will have two main sections: one for recording a new payment and one for viewing a student's payment history.
  - Both sections will use Streamlit forms and buttons to call the new methods on the manager object.
- **Check the Template give in:**
  - **gui/finance_pages.py**
- **Update gui/main_dashboard.py**: Import show_finance_page and replace the stub for the "Payments" page.

Checkpoint: Commit Your Progress
Save your file and commit your new function.

```
git add gui/finance_pages.py gui/main_dashboard.py
git commit -m "feat(gui): Build finance page for recording payments and viewing history"
```

Fragment 5.3: The Automated Test Suite

- **Mission:** To write a suite of automated unit tests using pytest that verify the correctness of your application's core logic. This is the ultimate safety net.
- **Your Task:**
  - Install pytest: pip install pytest.

    - Create a tests/ directory and tests/test_schedule_manager.py.
    - Write test functions that follow the "Arrange, Act, Assert" pattern. Each function should test one specific piece of logic.
- **Check the Template give in:**
    - **tests/test_schedule_manager.py**
- **How to Run:** In your terminal, simply run the command pytest. It will automatically discover and run your tests.

Checkpoint: Commit Your Progress
Save your file and commit this important update.

```
git add tests/
git commit -m "test(qa): Implement unit tests for core logic, completing PST5"
```

```
########################################
# 4  Push the new commit(s) up to GitHub
########################################
git push origin individual
#          |      |
#          |      └─ the branch name on GitHub
#          └──────── the remote repository (GitHub by default)
```

Instructions

- **Template files provided:** You will find a skeleton for every task (Fragment#_#.py)
- **Finish • Run • Test:** Complete each fragment, run it locally, and confirm it works.
- **Commit & push:** Test the complete application, then commit and push.
- **One README to rule them all:** Create **one** high-quality README.md at the root of your repo that explains
    - what each part does,
    - how to run and test the full program, any design choices or assumptions you made.
- **A clear, detailed README is worth marks, treat it like part of the assignment.**